

Documento de Técnicas Algorítmicas

Recursividad y Backtracking:

Problema de las 8 Reinas

Pérez Barahona Pedro Luis

12 de mayo de 2025

Resumen

En este reporte se analiza la técnica de *recursividad* y su aplicación al algoritmo de *backtracking* mediante el clásico problema de las 8 reinas. Se presenta la definición teórica de recursividad, el esquema general de backtracking y, finalmente, la implementación detallada en C++ y Python que enumera todas las soluciones válidas para un tablero de n reinas.

1. Introducción a la Recursividad

2. Algoritmo de Backtracking

3. Problema de las N-Reinas

3.1. Implementaciones en C++ y Python

A continuación se muestran las versiones completas en C++ y en Python del algoritmo de backtracking para el problema de las N-reinas. Ambas implementaciones utilizan comprobaciones en $O(1)$ para columnas y diagonales, y aplican recursividad con poda temprana.

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 void backtrack(int n, int row,
6               vector<int>& pos,
7               vector<bool>& col,
8               vector<bool>& diag1,
9               vector<bool>& diag2,
10              vector<vector<int>>& sols) {
11     if (row == n) {
12         sols.push_back(pos);
13         return;
14     }
15     for (int c = 0; c < n; ++c) {
16         int d1 = row - c + n;
17         int d2 = row + c;
18         if (col[c] || diag1[d1] || diag2[d2]) continue;
19         pos[row] = c;
20         col[c] = diag1[d1] = diag2[d2] = true;
21
22         backtrack(n, row + 1, pos, col, diag1, diag2, sols);
23
24         col[c] = diag1[d1] = diag2[d2] = false; // retroceso
25     }
26 }
27
28 int main() {
29     int n;
30     cout << "Ingrese n mero de reinas: ";
```

```

31     cin >> n;
32
33     vector<int> pos(n, -1);
34     vector<bool> col(n), diag1(2*n+1), diag2(2*n+1);
35     vector<vector<int>> solutions;
36
37     backtrack(n, 0, pos, col, diag1, diag2, solutions);
38
39     cout << "Total soluciones para n=" << n
40           << ": " << solutions.size() << endl;
41     for (size_t i = 0; i < solutions.size(); ++i) {
42         cout << "Soluci n " << i+1 << ": ";
43         for (int c: solutions[i]) cout << c << " ";
44         cout << endl;
45     }
46     return 0;
47 }

```

Listing 1: Implementación C++ del Problema de las N-Reinas

```

1 def solve_nqueens(n):
2     solutions = []
3     board = [-1] * n
4     cols, diag1, diag2 = set(), set(), set()
5
6     def backtrack(row):
7         if row == n:
8             solutions.append(board.copy())
9             return
10        for c in range(n):
11            d1, d2 = row - c, row + c
12            if c in cols or d1 in diag1 or d2 in diag2:
13                continue
14            board[row] = c
15            cols.add(c); diag1.add(d1); diag2.add(d2)
16
17            backtrack(row + 1)
18
19            cols.remove(c); diag1.remove(d1); diag2.remove(d2) # retroceso
20
21        backtrack(0)
22        return solutions
23
24 if __name__ == '__main__':
25     n = int(input("Ingrese n mero de reinas: "))
26     sols = solve_nqueens(n)
27     print(f"Total soluciones para n={n}: {len(sols)}")
28     for i, sol in enumerate(sols, 1):
29         print(f"Soluci n {i}: {sol}")

```

Listing 2: Implementación Python del Problema de las N-Reinas

4. Conclusiones

- La recursividad permite descomponer problemas complejos en casos más simples.
- El backtracking brinda un esquema de *divide y vencerás* con poda temprana de ramas inviables.
- La optimización con estructuras auxiliares (vectores en C++ o sets en Python) para columnas y diagonales mejora la eficiencia al reducir la comprobación a tiempo constante.

Aplicaciones reales

1. **Planificación de horarios y recursos.** Asignación de turnos laborales, aulas o salas de reuniones, donde cada asignación parcial se valida inmediatamente y se descarta si genera conflictos.

2. **Resolución de rompecabezas lógicos.** Juegos como Sudoku o generación de laberintos usan backtracking para rellenar celdas o rutas, retrocediendo ante soluciones parciales inviables.
3. **Testing y generación de configuraciones.** En pruebas de software y diseño de hardware, se exploran combinaciones de parámetros y se podan aquellas que no superan validaciones básicas.

5. Referencias

- Cormen, T. H., *Introduction to Algorithms*, MIT Press, 2009.
- Sedgewick, R., Wayne, K., *Algorithms*, Addison-Wesley, 2011.
- Weiss, M. A., *Data Structures and Algorithm Analysis in C+*, Pearson, 2014.